Justin
Seonyong
Ли э

Трансформеры: краткий обзор

Февраль 2021

Математический экскурс в архитектуру трансформеров и предшествующую литературу.

Цель этого поста — разобрать математические принципы, лежащие в основе трансформаторной архитектуры (https://en.wikipedia.org/wiki/Transformer_(machine_learning_model)), а также поделиться некоторыми полезными ресурсами и советами, основанными на моем опыте изучения этой архитектуры. Мы начнем с обзора литературы по преобразованию последовательностей, предшествовавшей появлению трансформера, а затем перейдем к основополагающей статье «Attention is All You Need» (https://arxiv.org/pdf/1706.03762.pdf) Васвани и др. (2017).

Содержание

- 1 До появления трансформеров
- 2 Внимание — это все, что вам нужно
- 3 Внимание на основе скалярного произведения
- 4 Многоголовое внимание
- 5 Применение многоголового внимания
- 6 Заключение

Перед Трансформерами [Вверху]

В то время как "*Внимание - это все, что вам нужно*" представила переломную архитектуру нейронной сети с обширными и растущими приложениями, обзор предыдущих исследований по преобразованию последовательностей очень поучителен. Это позволяет понять как стоящую за этим мотивацию, так и механизмы и методы, которые позволили разработать Transformer. Для отличного глубокого погружения в эту область с особым упором на машинный перевод я настоятельно рекомендую Prof. Учебник

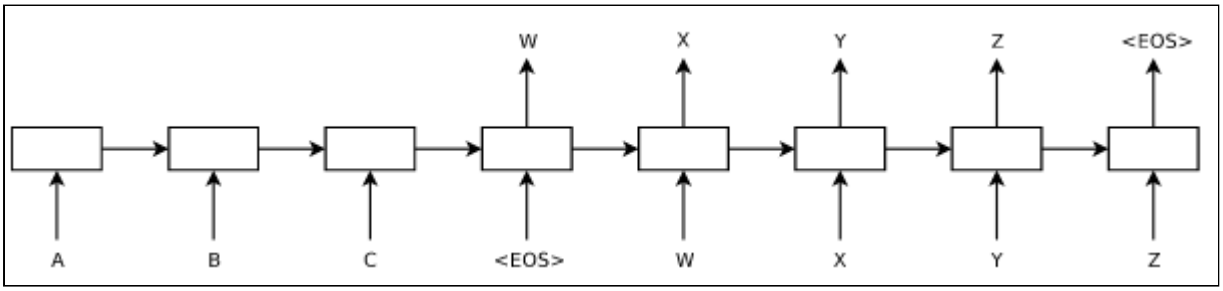
(https://arxiv.org/pdf/1703.01619.pdf) Грэма Нойбига. Он начинается с самых простых моделей, основанных на частоте и регрессии, и доходит до нейронных сетей с кодировщиком-декодировщиком и механизмом внимания. Большая часть этого раздела основана на различных частях руководства Нойбига или вдохновлена ими.

Статья Attention была написана в связи с проблемой машинного перевода. В соответствии с обозначениями, приведенными в руководстве, эту проблему можно сформулировать следующим образом: мы начинаем с исходной последовательности *[Math Processing Error]*, где каждый *[Ошибка математической обработки]* представляет собой отдельное слово из исходного словаря, например английского. Мы пытаемся предсказать перевод *[Math Processing Error]* на другой язык, то есть в целевую последовательность *[Ошибка обработки математического выражения]* состоит из слов *[Ошибка математической обработки]* из целевого словаря, например французского.

Прежде чем мы продолжим, несколько замечаний. Во-первых, обратите внимание, что длины *[Math Processing Error]* и *[Ошибка обработки математического выражения]* не обязательно совпадают. Это один из аспектов перевода, который делает его особенно сложным: человеческие языки не переводятся дословно. Во-вторых, отдельные «слова» *[Math Processing Error]* и *[Ошибка в вычислениях]* представлены в виде векторов, обычно с однократным кодированием.

С точки зрения теории вероятностей мы стремимся смоделировать распределение *[Math Processing Error]*Одно полезное наблюдение заключается в том, что мы можем разделить совместную вероятность *[Math Processing Error]* в виде произведения условных вероятностей:
[Ошибка обработки математического выражения] Такой подход к решению проблемы удобен для использования рекуррентных нейронных сетей, которые обеспечили большинство последних достижений в области машинного перевода вплоть до появления моделей на основе трансформеров.

В начале раздела 3, посвященного архитектуре модели, в статье отмечается, что «большинство конкурентоспособных моделей преобразования нейронных последовательностей имеют структуру **кодировщик-декодировщик**». Этот момент заслуживает внимания, поскольку Transformer также является моделью типа «кодировщик-декодировщик». Первые модели типа «кодировщик-декодировщик» для перевода были основаны на рекуррентных нейронных сетях и представлены почти одновременно в 2014 году в статьях Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation (https://arxiv.org/pdf/1406.1078.pdf) и Sequence to Sequence Learning with Neural Networks (https://arxiv.org/pdf/1409.3215.pdf). Концепция «кодировщик-декодировщик» в целом описывает ситуацию, в которой один процесс представляет или «кодирует» входные данные в виде одного вектора, а затем другой процесс «декодирует» этот вектор, получая желаемый результат. Эта концепция применима не только к обработке естественного языка, но и ко многим областям компьютерного зрения. Ниже представлена визуализация этого процесса из статьи Sequence to Sequence Суцкевера и др.



В общих чертах модели RNN с кодированием и декодированием работают следующим образом: пусть *[Math Processing Error]* Это модель рекуррентной нейронной сети, которая принимает входной вектор *[Ошибка обработки математического выражения]* и скрытое состояние *[Ошибка обработки математических выражений]*Эта рекуррентная нейронная сеть является «кодировщиком». Как и прежде, мы пытаемся предсказать *[Math Processing Error]* при условии *[Ошибка обработки математического выражения]*.

Для начала вычислим *[Math Processing Error]*, где *[Ошибка вычисления]* — это начальный вектор скрытого состояния. Он генерирует выходной вектор и вектор скрытого состояния *[Математическая обработка]*. Далее мы просто игнорируем выходной вектор и вычисляем *[Math Processing Error]*. Мы повторяем этот процесс до тех пор, пока не пройдемся по всему *[Ошибка обработки математического выражения]*. Цель состоит в том, чтобы «запустить» вектор скрытого состояния модели таким образом, чтобы к моменту, когда модель обработает все элементы исходной последовательности, скрытое состояние содержало всю информацию, необходимую для того, чтобы другая модель могла последовательно генерировать *[Math Processing Error]*. Соответственно, мы вводим рекуррентную нейронную сеть «декодера», *[Math Processing Error]*. Мы подаем на вход вектор скрытого состояния последнего кодера *[Ошибка вычисления математического выражения]* в качестве начального скрытого состояния декодера, а также первого элемента последовательности для декодирования (подробнее об этом через несколько абзацев).

Декодер выдает категориальное распределение по целевому словарю. Мы используем это распределение вместе с истинными значениями, чтобы вычислить вклад этого целевого элемента в общую потерю, которая представляет собой метрику, подходящую для сравнения распределений, например кросс-энтропию или дивергенцию Кульбака — Лейблера. Были проведены исследования, направленные на изменение этого аспекта процесса обучения, чтобы получить более обобщенные и менее самоуверенные модели. Один из таких методов, сглаживание меток, также используется в статье *Attention* во время обучения. Сглаживание меток было представлено в статье «Переосмысление архитектуры Inception для компьютерного зрения» (https://arxiv.org/pdf/1512.00567.pdf) в 2015 году. Оно предполагает использование средневзвешенного значения фактического «распределения» истин (категориального распределения с массой 1 в метке истинности) и дискретного равномерного распределения в качестве истин.

То, что подается на вход декодера, зависит от того, что мы делаем — обучаем модель или делаем вывод. При обучении мы подаем на вход целевые значения *[Math Processing Error]* по одному элементу за раз. Использование эталонных данных на каждом временном шаге,

в отличие от выходных данных, сгенерированных нейросетью, называется **принудительным обучением**. При этом вывод может осуществляться несколькими способами. Поскольку у нас нет эталонной целевой последовательности *[Math Processing Error]* во время логического вывода, и поскольку каждый элемент сгенерированного *[Math Processing Error]* Поскольку целевая последовательность зависит от предыдущих, нам нужно последовательно генерировать ее элементы. Целевая последовательность считается «готовой», когда модель определяет, что `<EOS>` тег является наиболее вероятным следующим токеном.

Это может показаться довольно простым, но есть некоторые нюансы. Самый простой метод генерации целевых последовательностей — это жадное декодирование, при котором каждое целевое предсказание *[Math Processing Error]* является argmax выходного распределения вероятностей на каждом шаге. Однако при жадном декодировании не гарантируется получение целевой *последовательности* с наибольшей вероятностью; то есть даже если на каждом временном шаге выбирать наиболее вероятный следующий элемент, это не исключает возможности существования другой последовательности, которая в целом более вероятна, чем решение, полученное методом жадного поиска. Чтобы учесть такую возможность, используется другой способ логического вывода — **поиск по лучу**, который предполагает одновременное изучение нескольких траекторий целевой последовательности и выбор последовательности с наибольшей вероятностью. Количество исследуемых траекторий — это гиперпараметр, называемый «шириной луча». *Attention* использует поиск по лучу для сравнительного анализа различных моделей после их обучения.

Один из недостатков кодера-декодера на основе рекуррентной нейронной сети заключается в том, что один вектор скрытого состояния может нести лишь ограниченный объем информации: чем длиннее входная последовательность, тем более зашумленной становится информация из ее предыдущих частей. Это ограничивает способность моделей на основе рекуррентных нейронных сетей делать точные прогнозы для длинных последовательностей. *Исследователи начали изучать возможность использования всех* векторов скрытого состояния, сгенерированных при кодировании, для декодирования целевой последовательности, а не только последнего скрытого состояния кодера. Ключевым нововведением в этом направлении стали **двунаправленные кодеры**, представленные в нейронном машинном переводе с совместным обучением выравниванию и переводу (https://arxiv.org/pdf/1409.0473.pdf) в 2016 году. Помимо того, что рекуррентная нейронная сеть обрабатывает исходную последовательность *[Math Processing Error]*При таком подходе добавляется еще одна рекуррентная нейронная сеть, которая проходит ту же последовательность в обратном направлении. На каждом временном шаге обе рекуррентные нейронные сети сохраняют свои скрытые состояния. Это означает, что после обработки всех слов в *[Math Processing Error]* После прохождения каждого из них с ним связаны два вектора скрытого состояния. Два вектора скрытого состояния объединяются в один, и все *[Math Processing Error]* Полученные векторы объединяются в одну матрицу.

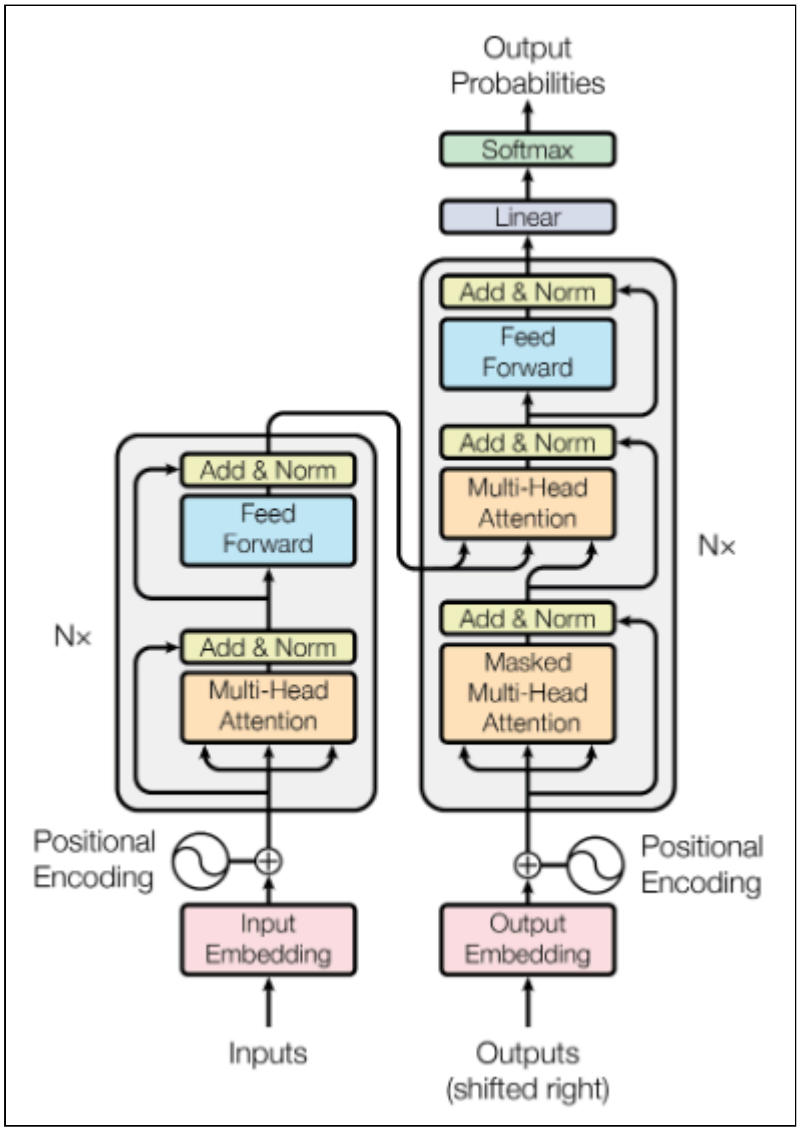
Но чтобы декодер мог использовать все скрытые состояния, эту матрицу необходимо

преобразовать в вектор одинакового размера — мы не можем передать в декодер скрытое состояние, размер которого зависит от количества элементов в исходной последовательности. Так появилось **внимание**. Механизм внимания преобразует этот набор скрытых состояний в взвешенную сумму составляющих его векторов, а вес будет частично зависеть от содержимого векторов. Одним из таких методов взвешивания было **внимание на основе скалярного произведения**, представленное в «Эффективных подходах к нейронному машинному переводу на основе внимания» (https://arxiv.org/pdf/1508.04025.pdf) в 2015 году.

Attention Is All You Need [Top]

Теперь, когда мы разобрались с предысторией, можно перейти к обсуждению статьи *Attention*, в которой авторы представляют новый тип архитектуры, позволяющий избежать многих проблем, связанных с моделями на основе рекуррентных нейронных сетей, в контексте машинного перевода. Несмотря на все достижения в области кодеров и декодеров на основе рекуррентных нейронных сетей, о которых говорилось выше, рекуррентные нейронные сети по-прежнему сложно распараллеливать из-за того, что они обрабатывают входные данные последовательно. Ключевое новшество этой статьи заключается в том, что использование рекуррентных нейронных сетей и их скрытых состояний полностью заменено операциями на основе внимания, которые более эффективны во многих проблемных ситуациях. Таким образом, архитектура Transformer устранила самое серьезное препятствие в процессе обучения предыдущих моделей, соответствующих последнему слову техники, и сделала возможным создание огромных моделей, обученных на таких же огромных наборах данных.

В следующих разделах мы более подробно рассмотрим различные компоненты трансформера. А пока начнем с общего обзора. Ниже представлена схема архитектуры из статьи.



Как уже упоминалось, Transformer — это модель кодер-декодер. Кодер состоит из *[Math Processing Error]* Блоки расположены слева, а декодер состоит из *[Ошибка обработки математических выражений]* блоки справа.

Во время обучения вводились слова *[Ошибка вычисления]* Все данные одновременно передаются в первый блок кодирования, а затем выходные данные этого блока передаются в следующий блок. Процесс повторяется до тех пор, пока все *[Math Processing Error]* Блоки кодирования обработали входные данные. Каждый блок состоит из двух компонентов: слоя Multi-Head Self-Attention, за которым следует полносвязный слой с активацией ReLU, который параллельно обрабатывает каждый элемент входной последовательности. После того как входные данные прошли через все блоки кодирования, мы получаем закодированное представление *[Math Processing Error]*. Как за слоем с несколькими головками внимания, так и за полносвязным слоем следует этап «Add & Norm» — «add» означает остаточную связь, которая добавляет входные данные каждого слоя к выходным, а «norm» означает нормализацию слоя (https://arxiv.org/pdf/1607.06450.pdf).

Перейдем к декодеру. Он состоит из трех этапов: слоя самовнимания с несколькими масками, слоя внимания с несколькими масками, соединяющего закодированное исходное

представление с декодером, и полносвязного слоя с активацией ReLU. Как и в кодировщике, за каждым слоем следует слой «Add & Norm». Декодер принимает все целевые слова *[Math Processing Error]* в качестве входных данных. Здесь следует отметить несколько ключевых отличий от кодировщика. Одно из них заключается в том, что входные данные для первой операции внимания в блоках декодера **маскируются**, отсюда и название слоя. Мы подробно рассмотрим это в разделе Multi-Head Attention, но в двух словах: это означает, что любое слово в целевом выводе может быть связано только с теми словами, которые были до него. Причина этого проста: во время *вывода* мы генерируем предсказанный перевод *[Math Processing Error]* Пословно, используя исходное предложение *[Ошибка обработки математического выражения]*. В процессе прогнозирования слова *[Ошибка в вычислениях]*, декодер имеет доступ к ранее сгенерированным словам. Однако он не может получить доступ к словам, которые следуют *[Ошибка обработки математического выражения]*, поскольку они еще не сгенерированы и зависят от выбора модели *[Math Processing Error]*. Маскирование во время обучения позволяет нам имитировать условия, с которыми модель столкнется при выводе данных.

Еще одно отличие от кодировщика — второй слой многоголового внимания, который также называют слоем внимания кодировщика-декодировщика. В отличие от слоев внимания в начале блоков кодировщика и декодировщика, этот слой не является слоем самовнимания. Самовнимание более подробно рассматривается в разделе Применение многоголового внимания, но на практике это означает, что внимание кодировщика-декодировщика связывает закодированное представление *[Math Processing Error]* с целевыми словами в *[Ошибка обработки математических выражений]*.

Scaled Dot-Product Attention [Top]

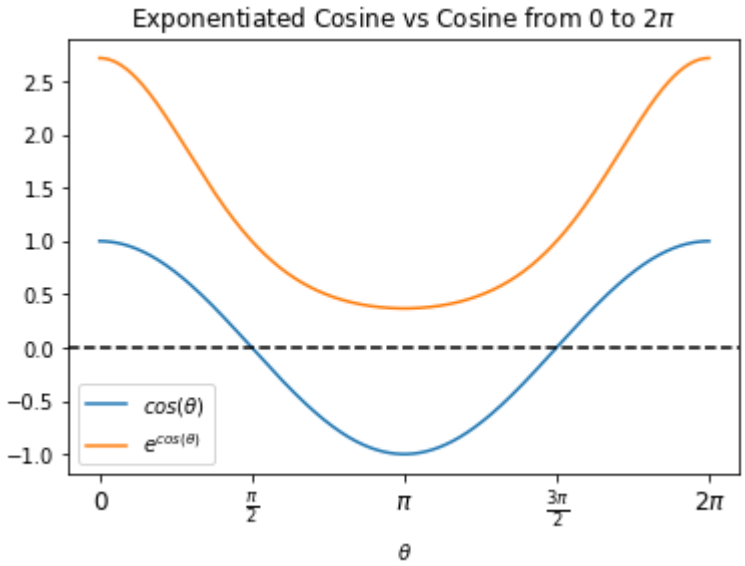
Теперь мы рассмотрим механизм Scaled Dot-Product Attention, описанный в статье, который представлен ниже. Эта функция используется во всех слоях внимания в трансформере. Для начала мы разберем математическую основу этой операции, чтобы понять, какие числа куда идут. Затем мы сосредоточимся на ее применении в статье. *[Ошибка обработки математического выражения]* Внимание на основе масштабированного скалярного произведения практически идентично вниманию на основе скалярного произведения, упомянутому ранее в статье Луонга и др. (2015). Единственное отличие заключается в том, что входные данные для функции softmax масштабируются с коэффициентом *[Math Processing Error]*. После того как мы разберемся, как работает метод внимания на основе скалярного произведения, мы вернемся к этому вопросу и рассмотрим его более подробно в конце этого раздела.

В статье и предшествующей ей литературе строки *[Math Processing Error]* называются "запросами", относящимися к *[Ошибке математической обработки]* «ключи» и, наконец, *[Ошибка математического преобразования]* «ценности». Пока что воспринимайте их просто как ярлыки; мы можем изучить их интерпретацию позже. Здесь *[Math Processing Error]*, *[Ошибка в математических вычислениях]* и *[Ошибка в вычислениях]*. Обратите внимание, что для выполнения алгебраических действий необходимо указать количество ключей и

значений *[Математическая обработка]* должны быть равны, но количество запросов *[Ошибка вычисления]* может варьироваться. Аналогичным образом, размерность ключей и запросов должна совпадать, а размерность значений может варьироваться.

Мы начнем наше исследование с выписывания отдельных строк *[Ошибка математической обработки]* и *[Ошибка в вычислениях]*, а затем выразим произведение *[Ошибка обработки математического выражения]* в терминах этих строк: *[Ошибка обработки математического выражения] [Ошибка обработки математического выражения]* Затем мы получаем веса внимания, разделив каждый элемент на *[Math Processing Error]* и применение функции softmax для каждой строки: *[Ошибка обработки математического выражения]* где для каждой строки *[Ошибка математического преобразования]*, в результате операции softmax, *[Ошибка обработки математического выражения]* Последний шаг — умножить эту матрицу на *[Math Processing Error]*: *[Ошибка математической обработки]* Вывод: механизм внимания приводит к вычислению ряда средневзвешенных значений по строкам *[Math Processing Error]*, где вес зависит от входных запросов и ключей. Каждый из *[Math Processing Error]* запросы в *[Ошибка обработки математического выражения]* приводит к получению определенной взвешенной суммы векторов значений. Следует отметить, что в этой конкретной процедуре нет обучаемых параметров — она полностью состоит из матричных и векторных операций.

Давайте увеличим масштаб и рассмотрим отдельную строку *[Математическая обработка]* весов внимания: *[Ошибка вычисления]* где *[Ошибка обработки математического выражения]* — константа нормализации: *[Math Processing Error]* Если посмотреть на то, как формируются веса, становится понятнее происхождение названий «запросы», «ключи» и «значения». Как и в хэш-таблице, эта операция выбирает нужные значения по соответствующим ключам, которые соответствуют друг другу. Ключи, которые мы ищем, обозначаются запросами. Мы можем выразить скалярное произведение между заданным ключом и запросом через угол *[Math Processing Error]* между ними: *[Ошибка вычисления]* Если на время отвлечься от величин, то на графике ниже мы увидим, что возведение в степень усиливает положительные значения косинуса и уменьшает отрицательные. Следовательно, чем ближе по углу ключ *[Math Processing Error]* и запрос *[Ошибка обработки математического выражения]* Чем больше человек сосредоточен на чем-то, тем сильнее его стремление к этому.



Если вернуться к контексту машинного перевода, то каждая строка с ключами, запросами и значениями в этом наборе данных представляет собой векторное представление элементов в последовательности. Один из недостатков моделей на основе рекуррентных нейронных сетей, который мы отметили, заключается в том, что им сложно использовать информацию об элементах, наблюдаемых в далеком прошлом. В более общем смысле, им сложно связывать последовательную информацию, которая находится далеко друг от друга. Такие методы, как внимание к скрытым состояниям и двунаправленные модели, были попытками решить эту проблему и послужили естественным переходом к методам, описанным в этой статье. Внимание на основе скалярного произведения позволяет эффективно и быстро связать каждый элемент последовательности со всеми элементами другой последовательности, а также со всеми остальными элементами той же последовательности.

В завершение этого раздела мы поговорим о коэффициенте масштабирования, который и дал название этому механизму внимания. Авторы Attention отмечают, что они делят входные данные для функции softmax на *[Math Processing Error]* чтобы смягчить влияние больших входных значений, которые привели бы к малым градиентам во время обучения. Авторы были обеспокоены тем, что длинные векторы ключей/запросов приведут к скалярным произведениям с большими значениями. Выбор коэффициента масштабирования можно объяснить сноской 4 в статье, а также тем фактом, что за исключением самых первых блоков кодировщика/декодировщика каждая входная матрица проходит этап нормализации слоя. Что касается того, почему большие значения функции softmax приводят к малым градиентам, то это можно объяснить с помощью некоторых математических выкладок. Начнем с некоторых определений: *[Math Processing Error]* Далее, пусть *[Ошибка обработки математического выражения]* представляют собой функцию softmax, входной вектор которой масштабируется с коэффициентом *[Math Processing Error]*. Обратите внимание, что если мы установим *[Ошибка математической обработки]*В итоге мы получаем исходный, немасштабированный softmax. *[Ошибка в расчётах]* Мы хотим рассмотреть производную функции масштабированного softmax по входному вектору *[Math Processing Error]*. Поскольку и входные, и выходные данные функции softmax являются векторными, на самом деле нам нужен якобиан: *[Math Processing Error]* *[Ошибка обработки математического выражения]* Если вычислить каждую частную производную и переставить

некоторые члены, то якобиан можно выразить следующим образом: *[Math Processing Error]* где *[Ошибка обработки математического выражения]* Функция проецирует векторы на диагональную матрицу (https://en.wikipedia.org/wiki/Diagonal_matrix), и *[Math Processing Error]* Это внешнее произведение (https://en.wikipedia.org/wiki/Outer_product). Приведенный ниже код создает визуализации масштабированных и немасштабированных матриц Якоби, вычисленных для случайно сгенерированных векторов различных размеров. Для заданной длины вектора *[Math Processing Error]*, каждый элемент взят из *[Math Processing Error]* чтобы соответствовать предположениям, изложенным в статье.

```
        jac_s = jacobian(s, C=1)
        jac_s_scaled = jacobian(s, C=scale)

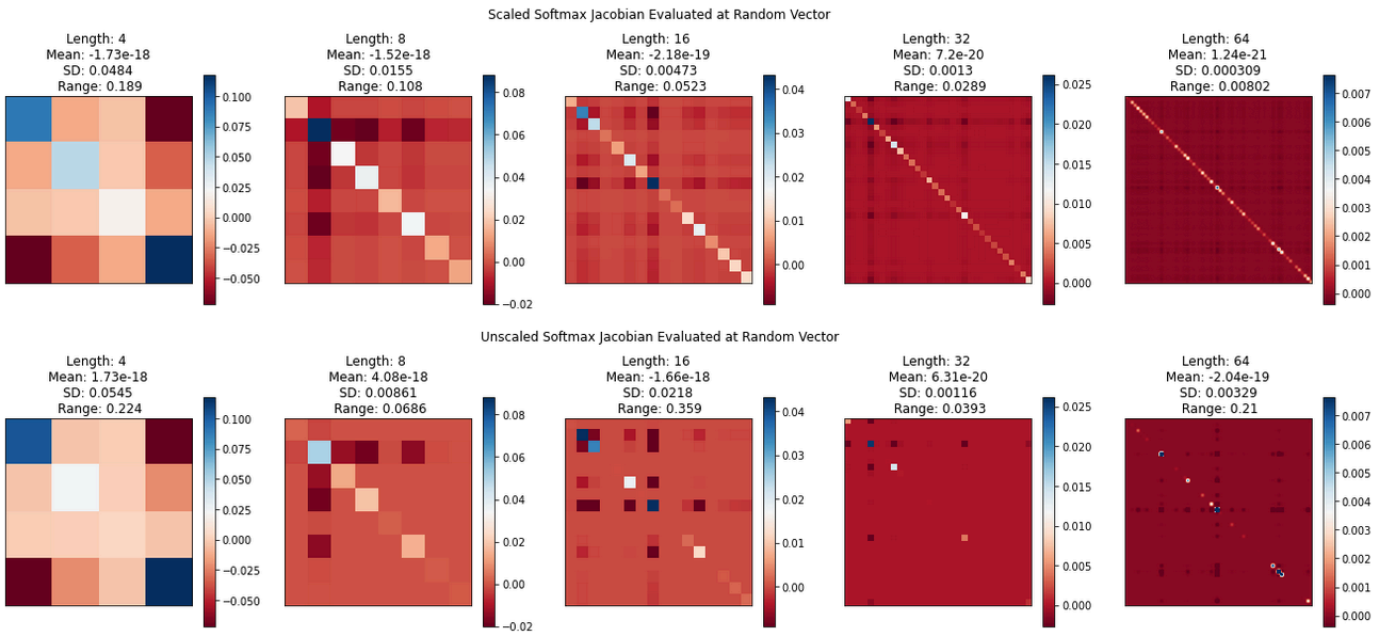
        # normalize to extrema of scaled Jacobian to better visualize
        # unscaled, which is often dominated by a handful of large values
        norm = Normalize(jac_s_scaled.min(), jac_s_scaled.max())

        plot_jacobian(jac_s_scaled, size, axes_scaled[n], norm=norm)
        plot_jacobian(jac_s, size, axes[n], norm=norm)

    plt.show()

sizes = [4, 8, 16, 32, 64] # some sample lengths for our inputs
generate_jacobian_comparison(sizes)
```

При выполнении функции `generate_jacobian_comparison()` выводятся данные, подобные приведенным ниже:



Повторные прогоны показывают, что, как правило, элементы немасштабированного якобиана функции softmax имеют более высокую дисперсию по сравнению с элементами масштабированного якобиана функции softmax, которые выглядят менее «пикообразными». Это неудивительно, если принять во внимание нашу формулу для якобиана, а также основные математические свойства функции softmax.

В функции softmax есть две простые операции, которые, так сказать, противоречат друг другу: возведение в степень и сложение. Экспоненциальные функции быстро меняются при увеличении входных данных — в конце концов, *[Math Processing Error]*. Если у нас есть вектор *[Ошибка обработки математического выражения]*Если возвести в степень его компоненты, получится *[Ошибка вычисления]*, и *[Ошибка обработки математического выражения]*. Если вместо этого возвести в степень каждый член выражения *[Math Processing Error]*В итоге мы получаем *[Ошибка обработки математического выражения]*, и *[Ошибка в математических вычислениях]*.

После возведения в степень каждого элемента входного вектора нам нужно вычислить коэффициент нормализации, сложив возведенные в степень значения. Вернемся к нашим примерам: *[Math Processing Error]* больше, чем *[Ошибка обработки математических выражений]* с коэффициентом *[Ошибка математической обработки]*. Таким образом, в выражении для нормализации второй элемент вносит вклад лишь в три раза меньший, чем первый. Но если мы вычислим то же самое с помощью *[Math Processing Error]*, мы можем быстро убедиться, что у первого элемента нет шансов — второй элемент вносит вклад в размере более *[Math Processing Error]* в несколько раз больше, чем член нормализации. В итоге *[Math Processing Error]*, но *[Ошибка обработки математического выражения]*. Вывод заключается в том, что незначительные различия между входными данными для функции softmax значительно усиливаются на соответствующих выходных данных.

Если вернуться к началу, то матрица Якоби снова состоит из двух частей: диагонализированного результата функции softmax и внешнего произведения результата функции softmax. Из-за своей структуры матрица Якоби наследует чувствительность функции softmax к экстремальным значениям входных данных, что приводит к тому, что одни элементы матрицы преобладают над другими. Хотя это свойство функции softmax может быть полезным в таких ситуациях, как классификация, оно не всегда желательно, когда операция выполняется глубоко внутри сети. Преимущество масштабирования заключается в том, что при делении входного вектора на скаляр теряется мало информации (поскольку исходную информацию можно восстановить, просто выполнив обратное действие, при условии, что арифметический переполнение не является проблемой). Однако свойства функции softmax означают, что при таком подходе якобианы в целом имеют более высокие значения во время обучения.

Многоголовое внимание [Top]

Многоголовое внимание — это расширение механизма внимания на основе скалярного произведения, при котором к запросам, ключам и значениям применяются линейные преобразования, в результате чего получается несколько наборов входных данных для функции внимания. Функция внимания вычисляется параллельно для каждого из этих наборов входных данных, а результаты объединяются в одну матрицу. Окончательный результат получается путем линейного преобразования объединенной матрицы для получения матрицы с желаемой размерностью. Авторы выражают эту мысль следующим

образом: *[Ошибка математического преобразования]* Каждый *[Ошибка обработки математического выражения]* Это результат применения алгоритма Scaled Dot-Product Attention к *[Математическая обработка ошибки]* набор преобразованных запросов, ключей и значений: *[Math Processing Error]* где *[Ошибка обработки математического выражения]*, *[Ошибка в математических вычислениях]*, и *[Ошибка в математических вычислениях]*. Кроме того, при наличии гиперпараметра *[Math Processing Error]* указывает на количество блоков внимания: *[Ошибка вычисления]*, *[Ошибка в математических вычислениях]*, *[Ошибка в математических вычислениях]*, и *[Ошибка в математических вычислениях]*.

Давайте быстро проверим, что умножение матриц работает. Во-первых, из предыдущего раздела мы знаем, что каждая матрица *[Math Processing Error]* будет содержать столько же строк, сколько *[Ошибка вычисления]*, и столько же столбцов, как *[Ошибка обработки математического выражения]*. Поскольку *[Ошибка обработки математических выражений]* и *[Ошибка в вычислениях]*, это означает *[Ошибка вычисления]*. Когда мы объединяем *[Ошибка обработки математического выражения]* Если сложить все это вместе, получится матрица в *[Math Processing Error]*. Умножение на *[Ошибка вычисления]* дает матрицу в *[Math Processing Error]*. В этом есть смысл — мы начали с *[Math Processing Error]* запросы в *[Ошибка обработки математического выражения]*, и в итоге у нас получилось *[Ошибка вычисления]* ответы в выводе *[Ошибка обработки математических выражений]* оператор.

Обратите внимание, что в каждом вычислении головы используется свое линейное преобразование для матриц ключа, запроса и значения. Каждое из этих преобразований изучается в процессе обучения. Читателям, знакомым с свёрточными нейронными сетями, я бы сравнил головы и их весовые матрицы с различными каналами и их обученными ядрами в свёрточных слоях. И те, и другие начинают работу с одних и тех же входных данных и генерируют несколько представлений этих данных одновременно, чтобы выявить их конкретные аспекты.

В следующем разделе мы рассмотрим три различных способа использования многоголового внимания в архитектуре Transformer.

Применение многоголового внимания [вверху]

Многоголовое внимание используется тремя способами в Attention: (1) вычисление самовнимания в блоках кодировщика, (2) вычисление самовнимания с маскированием в блоках декодера и (3) внимание кодер-декодер в блоках декодера.

Сначала мы рассмотрим механизм внутреннего внимания, при котором ключи, запросы и значения, передаваемые в функцию внимания, представляют собой одну и ту же матрицу. Использование механизма внутреннего внимания в блоках кодировщика и декодера в целом одинаково, но в последнем случае есть одна существенная особенность.

В слоях кодировщика мы выполняем следующую операцию: *[Math Processing Error]* где, подставив *[Ошибка в математических вычислениях]* Для матриц запроса, ключа и значения

в нашем вычислении для каждой головы механизма внимания мы получаем: *[Math Processing Error]* Блоки декодера делают то же самое, но с одним отличием: они выполняют один дополнительный шаг в функции внимания, чтобы гарантировать, что предыдущие элементы целевой последовательности могут взаимодействовать только с предыдущими элементами. Это достигается за счет **маскирования** входных данных для функции softmax при масштабированном внимании на основе скалярного произведения. Пусть *[Math Processing Error]*, *[Ошибка в математических вычислениях]*, и *[Ошибка в математических вычислениях]*. Обычно, учитывая *[Ошибка в математических вычислениях]* Определим его так же, как и выше: *[Ошибка математического выражения]* Поскольку декодер обрабатывает целевую последовательность, нам нужно сделать так, чтобы запросы не видели ключи, которые следуют за ними в последовательности. Чтобы понять почему, вернемся к разделу До появления трансформеров. Рассмотрим состояние модели во время вывода. Мы начинаем с исходной последовательности *[Math Processing Error]* и выполните поиск, чтобы сгенерировать целевую последовательность *[Ошибка в математических вычислениях]* элемент за элементом. Сначала модель получает доступ только к исходной последовательности, чтобы сгенерировать прогнозируемый первый целевой элемент *[Math Processing Error]*. На следующей итерации модель имеет *[Math Processing Error]* и *[Ошибка в вычислениях]* с которыми можно работать при составлении прогноза для *[Math Processing Error]*, и так далее, пока не будет сгенерирован тег `<EOS>` . Ни на одном из этапов генерации целевой элемент *[Ошибка обработки математического выражения]* есть ли у модели доступ к информации о будущих элементах *[Ошибка обработки математического выражения]* для *[Ошибка обработки математического выражения]*.

Маскирование входных данных целевой последовательности во время обучения имитирует эти условия. Во время обучения модель использует информацию из *[Math Processing Error]* строка входной матрицы для прогнозирования *[Math Processing Error]* целевой токен. В первой строке у нас есть только информация о нулевом или пустом токене, поскольку при генерации первого целевого элемента мы начинаем только с исходной последовательности (обратите внимание, что целевая последовательность сдвинута на одну позицию вправо — см. схему архитектуры). Во второй строке у нас есть информация о пустом токене и первом «настоящем» целевом токене для генерации прогнозов для второго целевого токена.

Один из способов замаскировать входные данные — просто добавить к аргументу softmax матрицу *[Math Processing Error]* состоит из нулей в нижнем треугольнике и *[Math Processing Error]*'s везде, кроме этого: *[Ошибка обработки математического выражения]* *[Ошибка обработки математического выражения]* Затем применение функции softmax к каждой строке приводит к тому, что все *[Ошибка вычисления]* Обнуляем ячейки, оставляя только действительные термины внимания.

Третье и последнее применение многоголового внимания в статье — это внимание «кодировщик-декодировщик», которое используется в блоках декодера непосредственно после слоя маскированного многоголового внимания для установления связи между исходной и целевой последовательностями. В отличие от самовнимания, где все три входных параметра представляют собой одну и ту же матрицу, здесь это не так. Для

справки ниже приведено описание многоголового внимания: *[Math Processing Error]* Что касается внимания при кодировании и декодировании, единственное отличие от предыдущего подхода заключается в том, что *[Math Processing Error]* поступает из слоя Masked Multi-Head Attention, а *[Math Processing Error]* и *[Ошибка в вычислениях]* — это закодированное представление *[Ошибка математического преобразования]*. Один из подходов заключается в том, что модель может задать вопрос о том, как каждая позиция в целевой последовательности соотносится с исходной, и получить представление об исходной последовательности, чтобы использовать его при генерации следующего слова в целевой последовательности.

Важно отметить, что все блоки декодера получают одни и те же данные от кодера. От первого до *[Math Processing Error]* Блок декодера, каждый из которых принимает закодированную исходную последовательность, использует ее в качестве ключей и значений.

Заключение [Начало]

В этом посте мы шаг за шагом рассмотрели ключевые компоненты архитектуры трансформера, представленной в статье «*Attention is All You Need*». Мы увидели, что многие составляющие трансформера были разработаны на основе предшествующих исследований в области преобразования последовательностей. Мы также рассмотрели различные механизмы внимания, описанные в статье, и, в частности, то, как они используют простые, но эффективные матричные операции для создания представлений входных последовательностей. В случае с блоками декодера мы увидели, как применяется многоголовое внимание для сопоставления закодированной исходной последовательности с каждым элементом целевой последовательности.

Понимание этих компонентов с точки зрения фундаментальных принципов — важнейший первый шаг на пути к изучению последующих моделей на основе трансформеров. Если говорить о классических примерах, то BERT (https://arxiv.org/pdf/1810.04805.pdf) — это серия блоков-кодировщиков, обученных с помощью тщательно разработанной целевой функции, учитывающей присущую кодировщику двунаправленность. Аналогичным образом каждая из моделей GPT (https://en.wikipedia.org/wiki/OpenAI#GPT) состоит из все более длинных серий блоков-декодировщиков.

Работа над трансформерами продолжается стремительными темпами как в области исследований, так и в области разработок. Исследования моделей на основе трансформеров по-прежнему увлекательны: недавние исследования показали, что трансформеры эффективны при решении различных задач компьютерного зрения, в рамках генеративно-состязательных сетей (https://arxiv.org/pdf/2102.07074.pdf), а также в областях, где обработка естественного языка связана с компьютерным зрением, таких как проект DALL-E (https://openai.com/blog/dall-e/) от OpenAI. Также ведутся исследования, направленные на то, чтобы сделать трансформеры меньше и/или эффективнее.

Что касается разработки, исследователи и практики продолжают выпускать большие модели-трансформеры, предварительно обученные на больших массивах данных, которые можно использовать как есть или дообучить для решения более конкретных задач. Такие компании, как Hugging Face (https://huggingface.co/), разрабатывают инструменты и процессы с открытым исходным кодом, чтобы упростить использование этих моделей для исследователей и инженеров. И по мере того, как с каждым новым достижением сокращается разрыв между исследованиями и внедрением, значение архитектуры трансформеров будет только расти.

© 2026 Джастин С. Ли